CHAPTER 22

AI ENGINEER ROADMAP · AGENTIC AI USING LANGGRAPH · STUDY NOTES

# Implementing Short-Term Memory in LangGraph

Checkpointers, threads, PostgreSQL-backed persistence, and the token-trimming, deletion and summarization techniques that keep a long-running conversation inside the LLM’s context window.

## WHAT THIS CHAPTER COVERS

- Checkpointers & thread_id
- InMemorySaver implementation
- Thread isolation
- RAM volatility problem
- PostgresSaver via Docker
- Context overflow problem
- Trimming with trim_messages
- Deletion with RemoveMessage
- Summarization workflow

# Implementing Short-Term Memory in LangGraph

**RECAP — CHAPTER 21**

Chapter 21 established the theory: LLMs are stateless parametrized functions with no intrinsic memory, so *short-term memory (STM)* is engineered by concatenating the full conversation buffer into every call, and *long-term memory* survives across conversations. This chapter implements STM concretely inside LangGraph using checkpointers and threads, persists it to a real database, and fixes STM's context-overflow weakness with trimming and summarization.

## 1. Checkpointers & Threads — The Core Mechanism

### Definition

A *checkpointer* is a LangGraph component that saves a snapshot of the graph's entire state at every superstep (i.e. after every node finishes executing). A *thread* (identified by a `thread_id`) is a label that groups a sequence of checkpoints together, so that one particular conversation's history can be retrieved independently of every other conversation running on the same compiled graph.

### Why It Exists

Without a checkpointer, a compiled LangGraph graph behaves exactly like a stateless function: each `graph.invoke()` call starts from a blank state. Checkpointers give the graph a place to persist state between invocations, which is the foundation every short-term-memory implementation in LangGraph builds on.

### How It Works

1. A checkpointer instance (e.g. `InMemorySaver()`) is created.
2. The graph is compiled with `builder.compile(checkpointer=checkpointer)`.
3. A config dictionary is created containing a `thread_id`: `{'configurable': {'thread_id': 'thread-1'}}`.
4. Every `graph.invoke(message, config)` call passes this config, so the checkpointer knows which thread's history to load before the call and which thread to save the new state under after it.
5. Changing the `thread_id` starts a logically separate conversation — its checkpoint history is empty until messages are sent under that new id.

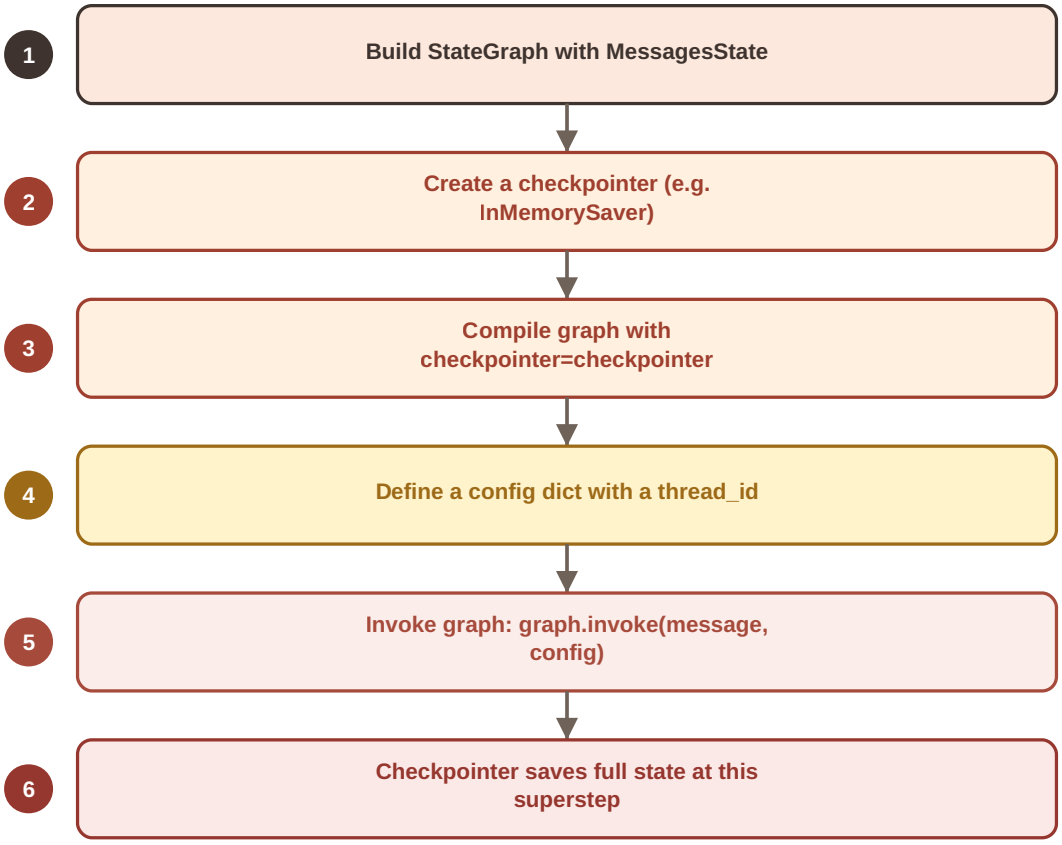



*Figure 22.1 — The six-step mechanism that turns a stateless graph into one with short-term memory.*

**IMPORTANT**

A checkpointer alone does nothing without a `thread_id`. The checkpointer decides *where* state is stored (RAM, Postgres, etc.); the thread_id decides *which conversation's* state is being read or written. Both are required together.

**↻ Quick Revision — Checkpointers & Threads**

- A checkpointer saves the full graph state at every superstep.
- A thread_id groups checkpoints into one logical conversation.
- Compile with checkpointer=... ; pass config={'configurable':{'thread_id':...}} on every invoke().
- Without a checkpointer, every invoke() call is a fresh, memory-less conversation.

## 2. Implementing STM with InMemorySaver

### Behavior Without Short-Term Memory

Before adding a checkpointer, a simple one-node `MessagesState` graph is invoked twice: first with "Hi! My name is Saurabh.", then with "What is my Name?". Because no state is carried between calls, the second invoke has no record of the first, and the model replies that it cannot know the user's name.

```
from langgraph.graph import StateGraph, START, MessagesState
from langchain_groq import ChatGroq
from dotenv import load_dotenv

load_dotenv()
model = ChatGroq(model = 'openai/gpt-oss-safeguard-20b')

def call_model(state: MessagesState):
    response = model.invoke(state['messages'])
    return {'messages':[response]}

builder = StateGraph(MessagesState)
builder.add_node('call_model', call_model)
builder.add_edge(START, 'call_model')

graph = builder.compile()  # no checkpointer -> no memory

graph.invoke({'messages':[{'role':'user','content':'Hi! My name is Saurabh.'}]})
graph.invoke({'messages':[{'role':'user','content':'What is my Name?'}]})
# -> model has no idea; each call is an independent conversation
```

## Adding the Checkpointer

The same graph structure is rebuilt, this time compiled with an `InMemorySaver` checkpointer and invoked with a config carrying a `thread_id`. Now the second question is answered correctly because the full message history for `thread-1` is reloaded before the model is called.

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer = checkpointer)

config = {'configurable': {'thread_id': 'thread-1'}}

graph.invoke({'messages':[{'role':'user','content':'My name is Saurabh.'}]}, config)
graph.invoke({'messages':[{'role':'user','content':'What is my name?'}]}, config)
# -> "Your name is Saurabh."  (full history replayed from the checkpointer)

# Inspecting the stored state directly:
snap = graph.get_state(config)
vals = snap.values
for m in vals.get('messages', []):
    print('-', type(m).__name__, ':', m.content)
```

## Inspecting State

`graph.get_state(config)` returns a snapshot object whose `.values` dictionary mirrors the graph's state schema (here, `messages`). This is the standard way to verify exactly what a given thread currently remembers, without issuing another `invoke()` call.

**BEST PRACTICE**

Always check a thread's state with `graph.get_state(config)` during development — it is the fastest way to confirm whether a checkpointer and thread_id combination is actually persisting what you expect.

**↻ Quick Revision — InMemorySaver**

- InMemorySaver stores every checkpoint in RAM, keyed by thread_id.
- compile(checkpointer=InMemorySaver()) is the minimal change needed to add STM.
- graph.get_state(config) returns a snapshot with a .values dict matching the state schema.
- Without a thread_id in config, the checkpointer cannot route state to the right conversation.

# 3. Thread Isolation

## Definition

Thread isolation is the guarantee that two different `thread_id` values against the same compiled graph and checkpointer never see each other's message history. Each thread_id is effectively an independent key into the checkpointer's storage.

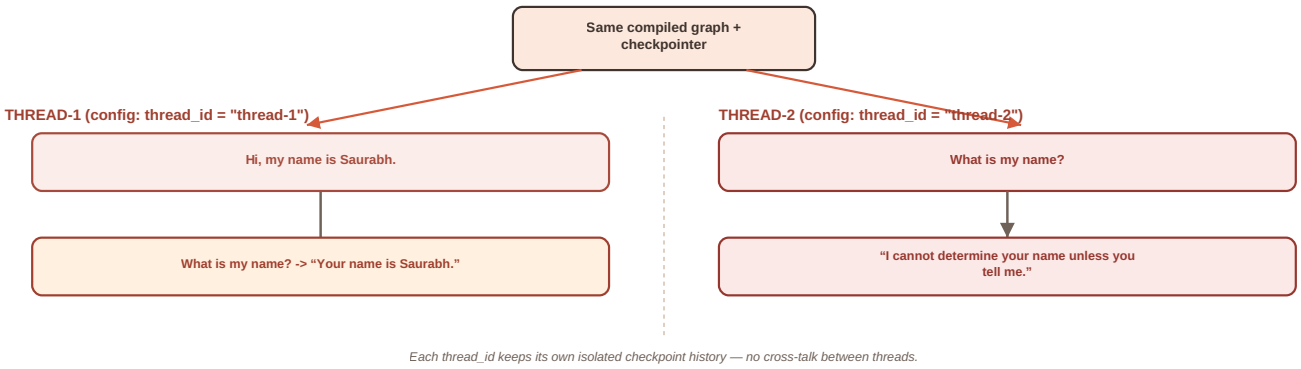



*Figure 22.2 — Thread-1 and Thread-2 share the same compiled graph but maintain completely separate conversation buffers.*

## Example

Using the same compiled graph from Section 2, switching the config to `{'configurable': {'thread_id': 'thread-2'}}` and asking "What is my name?" produces "I am sorry, I do not have the ability to know your name" — even though `thread-1` already knows the user's name. The new thread starts with a blank checkpoint history.

**COMMON MISTAKE**

Hardcoding a single `thread_id` for every user of an application means every user shares the same memory. Production chatbots generate a fresh `thread_id` per conversation (commonly via `uuid.uuid4()`) so that each user's history stays isolated.

**↻ Quick Revision — Thread Isolation**

- Different thread_id values on the same graph produce completely independent conversations.
- Switching thread_id is equivalent to starting a brand-new chat with the same bot.
- Thread isolation is what lets one deployed chatbot serve many simultaneous users safely.

# 4. The Persistence Problem

## Problem It Solves

`InMemorySaver` stores every checkpoint in the Python process's RAM. RAM is volatile: as soon as the program restarts (or the process is killed and reloaded), every checkpoint for every thread is permanently lost. Querying `get_state(config)` for a thread that had messages in a previous run returns an empty state after a restart.

## Why It Matters in Production

A production system deployed for real users cannot tolerate losing every conversation on a server restart or deploy. LangGraph's documentation recommends backing the checkpointer with a production-grade database instead — PostgreSQL is the officially recommended choice.

**IMPORTANT**

Trimming and deletion of individual messages (Sections 7–9) are unrelated to this RAM-volatility problem — those techniques control how much of an *already-persisted* history is sent to the LLM. Persistence (this section) is about whether the history survives a process restart at all.

**↻ Quick Revision — The Persistence Problem**

- InMemorySaver keeps all checkpoints only in RAM.
- A process restart clears RAM, so all threads' histories are lost with InMemorySaver.
- LangGraph's own documentation recommends PostgreSQL for production checkpoint storage.

# 5. Persisting STM with PostgreSQL

## Setup Options

Two practical ways to get a PostgreSQL instance running locally were discussed: installing PostgreSQL directly on the machine (prone to installation issues), or running it inside a Docker container via a `docker-compose.yaml` file (the chosen, more reliable approach).

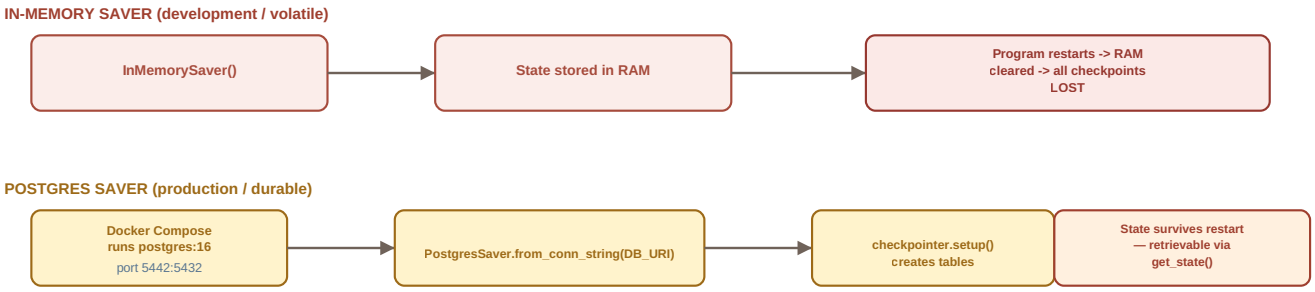



*Figure 22.3 — InMemorySaver (volatile) contrasted with PostgresSaver backed by a Dockerized Postgres instance (durable).*

## 1. DOCKER COMPOSE CONFIGURATION

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: postgres
    ports:
      - "5442:5432"
```

Run with `docker compose up -d`, then verify the container is running with `docker ps`. Host port `5442` is mapped to the container's internal Postgres port `5432`.

## 2. REQUIRED PYTHON DEPENDENCIES

```
pip install langgraph-checkpoint-postgres
```

## 3. IMPLEMENTATION

The same `StateGraph(MessagesState)` / `call_model` node / `add_edge(START, 'call_model')` structure is reused unchanged. The only new piece is wrapping compilation and invocation inside a `PostgresSaver.from_conn_string(...)` context manager, and calling `checkpointer.setup()` once to create the required tables.

```
from langgraph.checkpoint.postgres import PostgresSaver

DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres"

with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    # Run ONCE, creates tables
    checkpointer.setup()

    graph = builder.compile(checkpointer = checkpointer)

    # Thread-1 remembers
    t1 = {'configurable': {'thread_id': 'thread-1'}}
    graph.invoke({'messages':[{'role':'user','content':'Hi, my name is Saurabh.'}]}, t1)
    out1 = graph.invoke({'messages':[{'role':'user','content':'What is my name?'}]}, t1)
    print('Thread-1:', out1['messages'][-1].content)
```

## 4. VERIFYING PERSISTENCE AFTER A RESTART

After deliberately restarting the Python kernel/shell (simulating a production process restart), the graph is rebuilt and, without invoking any new message, the state for `thread-1` is fetched directly from Postgres:

```
DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres"
t1 = {'configurable': {'thread_id': 'thread-1'}}

with PostgresSaver.from_conn_string(DB_URI) as cp:
    g = builder.compile(checkpointer = cp)

    snap = g.get_state(t1)   # pulls from Postgres, not RAM
    msgs = snap.values.get('messages', [])
    print('Last Message: ', msgs[-1].content if msgs else None)
# -> "Your name is Saurabh."  (survived the restart)
```

**BEST PRACTICE**

Call `checkpointer.setup()` only once per database (it creates the checkpoint tables). Running it again on an already-set-up database is safe but unnecessary — it is typically called the first time a deployment connects to a fresh Postgres instance.

### ↻ Quick Revision — PostgreSQL Persistence

- PostgresSaver.from_conn_string(DB_URI) is used as a context manager around compile() and invoke().
- checkpointer.setup() creates the necessary tables — run once per database.
- The graph structure and node code are completely unchanged versus InMemorySaver — only the checkpointer swaps.
- State fetched via get_state() after a full process restart proves persistence genuinely survives restarts.

# 6. The Context Overflow Problem

## Definition

Every LLM has a fixed *context window*: the maximum number of tokens it can process in a single call. Because short-term memory works by concatenating the entire past conversation into every new call, a long-running conversation can eventually push the total input token count past the model's context window.

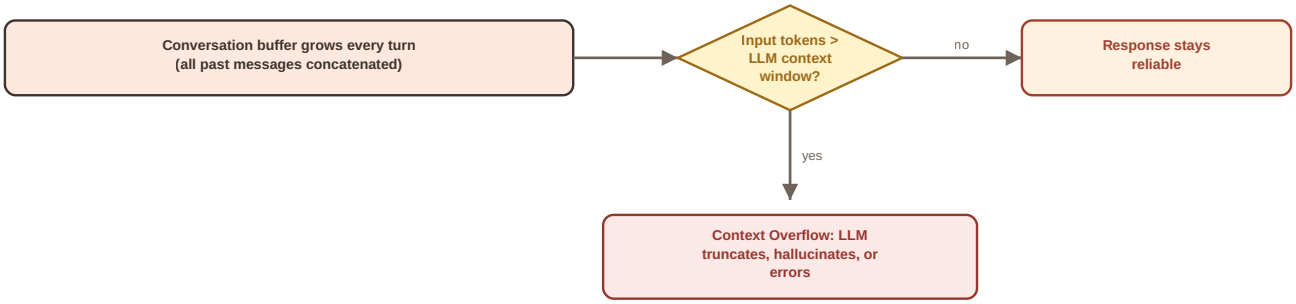



*Figure 22.4 — Every turn grows the conversation buffer; once it exceeds the model's context window, reliability breaks down.*

## Why It Matters

When input tokens exceed the context window, the model's response quality degrades unpredictably — it may hallucinate, truncate reasoning, or return malformed output. The practical rule is that input tokens should always stay well under the context window, not just barely inside it.

> **REAL WORLD INSIGHT**
>
> This is exactly the same context-overflow problem introduced conceptually in Chapter 21's discussion of STM's limitations. The next three sections implement the two production fixes for it: trimming and summarization (deletion is covered as the mechanism underlying both).

### ↻ Quick Revision — Context Overflow

- Every LLM has a maximum context window measured in tokens.
- Naively concatenating full conversation history risks exceeding that window as conversations grow.
- Overflow causes unreliable output: hallucination, truncation, or errors.
- Fix: keep input tokens well below the context window at all times, using trimming or summarization.

# 7. Trimming Strategy

## Definition

*Trimming* sets a maximum token budget (`max_tokens`). Before every LLM call, LangChain's `trim_messages()` function checks the total token count of the conversation; if it is under the budget, everything is sent unchanged. If it exceeds the budget, only the most recent messages that fit within the limit are kept — everything older is excluded from what is sent to the model.

## How It Works

1. A `MAX_TOKENS` constant is defined (e.g. `150`).
2. Inside the node, `trim_messages(state['messages'], strategy='last', token_counter=count_tokens_approximately, max_tokens=MAX_TOKENS)` is called before invoking the LLM.
3. `strategy='last'` means the trimmer keeps messages counting backward from the most recent one, stopping as soon as adding one more message would exceed `max_tokens`.
4. Only the trimmed message list is sent to `model.invoke()` — the full, untrimmed list still lives in the graph's state and in the checkpointer.

```
from langchain_core.messages.utils import trim_messages, count_tokens_approximately

MAX_TOKENS = 150

def call_model(state: MessagesState):
    messages = trim_messages(
        state['messages'],
        strategy = 'last',
        token_counter = count_tokens_approximately,
        max_tokens = MAX_TOKENS
    )

    response = model.invoke(messages)
    return {'messages': [response]}
```

Running this across several turns shows the token count climbing (10 → 36 → 122) with no trimming while it stays under 150. Once the next turn would push the total above 150, `trim_messages` drops the older messages and keeps only what fits — in the lecture's run, this meant the model lost the earlier "my name is Saurabh" message entirely and could no longer answer "What is my name?".

> **IMPORTANT**
>
> Trimming does not delete anything from the graph's state or from the checkpointer's storage — the full history is still there. Trimming only controls what is *shown to the LLM* on this particular call.

**COMMON MISTAKE**

`count_tokens_approximately` is a heuristic, not an exact tokenizer count — its numbers (e.g. counting a single word like 'user' as a token) will not exactly match a model's real tokenizer. Treat the numbers as approximate, not precise.

**↻ Quick Revision — Trimming**

- trim_messages(messages, strategy='last', token_counter=..., max_tokens=...) keeps the most recent messages under a token budget.
- Older messages beyond the budget are excluded from the LLM call, not deleted from state.
- count_tokens_approximately is an approximate heuristic, not an exact tokenizer.
- max_tokens must be tuned experimentally per application.

# 8. Message Deletion

## Definition

Unlike trimming (which only filters what is sent to the LLM), *deletion* permanently removes messages from the graph's state using LangChain's `RemoveMessage` object, identified by each message's `.id`. Deletion is the mechanism that summarization (Section 9) builds on.

## Implementation

A second node, `delete_old_messages`, runs after the chat node on every turn. If the message list has grown past 10 messages, it builds a list of `RemoveMessage` objects for the earliest 6 and returns them as the new `messages` state update — LangGraph's `add_messages` reducer interprets `RemoveMessage` entries as deletions rather than appends.

```
from langchain.messages import RemoveMessage

def chat(state: MessagesState):
    response = model.invoke(state['messages'])
    return {'messages': [response]}

def delete_old_messages(state: MessagesState):
    msgs = state['messages']

    # If more than 10 messages, delete the earliest 6
    if len(msgs) > 10:
        to_remove = msgs[:6]
        return {'messages': [RemoveMessage(id = m.id) for m in to_remove]}

    return {}

builder = StateGraph(MessagesState)
builder.add_node('chat', chat)
builder.add_node('cleanup', delete_old_messages)

builder.add_edge(START, 'chat')
builder.add_edge('chat', 'cleanup')
builder.add_edge('cleanup', '__end__')

graph = builder.compile(checkpointer = InMemorySaver())
```

After seven invocations (which would normally accumulate 14 messages — 7 user, 7 AI), checking `graph.get_state(config)` shows only **8** messages remain: the earliest 6 were removed by the `cleanup` node the moment the count crossed 10.

> **INTERVIEW TIP**
>
> Be ready to explain the key distinction: trimming is a per-call, non-destructive filter; deletion via `RemoveMessage` is a permanent, destructive edit to the state itself. Interviewers often probe whether you understand that the two solve the same symptom (long context) through fundamentally different mechanisms.

### ↻ Quick Revision — Deletion

- RemoveMessage(id=m.id) permanently deletes a message from state, unlike trimming.
- A cleanup node conditionally removes the oldest messages once a count threshold is crossed.
- add_messages (the MessagesState reducer) interprets RemoveMessage entries as deletions.
- Deletion is destructive — once removed, a message is gone from the checkpointer too.

## 9. Summarization Strategy

### Definition

*Summarization* improves on trimming by never fully discarding old context. Instead of ignoring older messages, they are sent to the LLM to be condensed into a running summary; that summary is kept and combined with only the most recent N messages on every subsequent call. The old verbatim messages are then deleted from state using `RemoveMessage` — summarization and deletion always operate together.

### Problem It Solves

Trimming's weakness is that it *completely* ignores everything older than its token budget allows, which can break conversational coherence in real-world scenarios where older context is still relevant. Summarization keeps at least a condensed version of that older context available to the model at every turn.

### How It Works

A custom state schema adds a `summary` field on top of `MessagesState`:

```
class ChatState(MessagesState):
    summary: str
```

Two nodes cooperate:

1. **chat_node** — if a summary already exists in state, it is prepended as a system message before the rest of the conversation; otherwise only the raw messages are sent.
2. **summarize_conversation** — triggered only when the message count exceeds 6 (via a conditional edge using `should_summarize`). It builds a prompt (different wording depending on whether a summary already exists, so new content is merged into it rather than overwriting it), asks the LLM to generate/extend the summary, then deletes every message except the most recent 2 using `RemoveMessage`.




*Figure 22.5 — Trimming, deletion, and summarization compared side by side.*

```
def chat_node(state: ChatState):
    messages = []
    if state.get('summary'):
        messages.append({
            'role': 'system',
            'content': f"Conversation Summary: \n{state['summary']}"
        })
    messages.extend(state['messages'])

    response = model.invoke(messages)
    return {'messages': [response]}

def summarize_conversation(state: ChatState):
    existing_summary = state.get('summary')

    if existing_summary:
        prompt = (f"Existing Summary: \n\n {existing_summary}\n"
                  "Extend the existing conversation using the messages above.")
    else:
        prompt = 'Generate the summary of the above messages conversation.'

    messages_for_summary = state['messages'] + [HumanMessage(content = prompt)]
    response = model.invoke(messages_for_summary)

    # Keep only last 2 messages as verbatim
    messages_to_delete = state['messages'][:-2]

    return {
        'summary': response.content,
        'messages': [RemoveMessage(id = m.id) for m in messages_to_delete]
    }

def should_summarize(state: ChatState):
    return len(state['messages']) > 6

builder = StateGraph(ChatState)
builder.add_node('chat', chat_node)
builder.add_node('summarize', summarize_conversation)

builder.add_edge(START, 'chat')
builder.add_conditional_edges(
    'chat', should_summarize,
    {True: 'summarize', False: '__end__'}
)
builder.add_edge('summarize', '__end__')

graph = builder.compile(checkpointer = InMemorySaver())
```

## Example Run

Across four turns (Quantum Physics → Einstein's relation to it → his famous works → special relativity), the summary stays empty while the message count is 2, 4, then 6. On the fourth turn the total reaches 8 messages (> 6), triggering

`summarize_conversation`: the state collapses down to the 2 most recent messages plus a freshly generated summary ("The conversation above discusses Albert Einstein's contribution to physics…"), which is then prepended as a system message on the next call.

**IMPORTANT**

Deleting the old messages after generating a summary is deliberate, not incidental: keeping both the old verbatim messages *and* their summary in state at the same time would duplicate context and confuse the model about what is current.

**↻ Quick Revision — Summarization**

- Summarization = condense old messages into a running summary + keep only the newest N verbatim.
- A custom state schema is needed to hold the extra 'summary' field alongside messages.
- A conditional edge (should_summarize) triggers the summarize node only past a message-count threshold.
- Old messages are deleted via RemoveMessage immediately after being folded into the summary.
- The summary is re-injected as a system message on every subsequent chat_node call.

## 10. Trimming vs. Summarization

| Aspect | Trimming | Summarization |
|---|---|---|
| Older messages | Completely ignored once over budget | Condensed into a summary, context preserved |
| State impact | Non-destructive — full history stays in state | Destructive — old messages deleted after summarizing |
| Extra cost | None (just a token-count check) | One extra LLM call to generate/extend the summary |
| Best for | Simple apps where only recent context matters | Long-running assistants where earlier context stays relevant |
| Trigger | Token count > max_tokens | Message count > threshold (conditional edge) |

**BEST PRACTICE**

Most production chatbots use summarization rather than plain trimming, precisely because real conversations often circle back to earlier topics. Trimming is simpler to reason about and cheaper (no extra LLM call), so it remains a reasonable default for short-lived or low-stakes interactions.

### ↻ Quick Revision — Trimming vs. Summarization

- Trimming is cheaper and simpler but permanently loses access to older context for the LLM.
- Summarization costs one extra LLM call per trigger but retains condensed context indefinitely.
- Both are triggered by a threshold check (tokens for trimming, message count for summarization here).
- The choice depends on how often older context genuinely matters to the application.

## K. Key Takeaways

- **Checkpointers** save full graph state at every superstep; **threads** (via thread_id) isolate one conversation's checkpoints from another.
- **InMemorySaver** implements STM with zero extra infrastructure, but stores everything in volatile RAM.
- **Thread isolation** lets one compiled graph safely serve many independent conversations simultaneously.
- The **persistence problem**: RAM-backed checkpointers lose all history on process restart — unacceptable in production.
- **PostgresSaver**, run via Docker Compose and set up with checkpointer.setup(), persists checkpoints durably across restarts with no change to graph logic.
- The **context overflow problem** arises because STM concatenates the entire conversation buffer into every call, risking exceeding the LLM's context window.
- **Trimming** (trim_messages, strategy='last', max_tokens) keeps only the most recent messages under a token budget — non-destructive to state.
- **Deletion** (RemoveMessage(id=...)) permanently removes messages from state and underlies both cleanup nodes and summarization.
- **Summarization** condenses older messages into a running summary (kept in a custom state field) while deleting the originals, preserving context more gracefully than trimming.

## R. Revision Sheet

### Checkpointer

Component that saves the graph's full state at every superstep; required for any form of memory in LangGraph.

### Thread / thread_id

A label grouping checkpoints into one isolated conversation history on a shared compiled graph.

### InMemorySaver

RAM-backed checkpointer; simplest to set up but loses all state on process restart.

### PostgresSaver

Database-backed checkpointer; set up via from_conn_string() + setup(); survives restarts.

### Context Overflow

When concatenated conversation history exceeds the LLM's context window, degrading response quality.

### Trimming

trim_messages() keeps only the most recent messages under a token budget; non-destructive to state.

### RemoveMessage

LangChain object used with a message's .id to permanently delete it from state via the add_messages reducer.

### Summarization

Condenses older messages into a running summary field, then deletes the originals; retains context better than trimming.

## I. Interview Questions

**Q1.** What does a checkpointer do in LangGraph, and at what point in execution does it save state?

**Q2.** What is a thread_id, and what would happen if two different users shared the same thread_id in a deployed chatbot?

**Q3.** Why is InMemorySaver unsuitable for a production deployment, even though it is the simplest checkpointer to set up?

**Q4.** Walk through the steps required to switch a graph from InMemorySaver to PostgresSaver.

**Q5.** What is the context overflow problem, and why does naive short-term memory make it worse as a conversation grows?

**Q6.** Explain how trim_messages() with strategy='last' decides which messages to keep.

**Q7.** Does trimming delete messages from the graph's state? Justify your answer.

**Q8.** What is RemoveMessage used for, and how does the add_messages reducer treat it differently from a normal message?

**Q9.** Describe how a summarization node decides when to trigger, and what two things it returns.

**Q10.** Compare trimming and summarization in terms of cost, state impact, and conversational coherence.

## F. Further Reading

- LangGraph documentation — Persistence (checkpointers, threads, state management)
- LangGraph documentation — Memory (short-term vs. long-term memory guides)
- LangChain documentation — trim_messages() and message utilities
- langgraph-checkpoint-postgres package documentation (PostgresSaver)
- PostgreSQL official documentation — Docker image reference